

10 - 子 Agent 与定时任务

阅读时间：约 1.5 小时
前置知识：09 - 多平台接入
学习目标：理解 SubAgent 后台任务机制、Cron 定时调度系统、Heartbeat 心跳服务，掌握并发任务设计

目录

- 10.1 为什么需要子 Agent 和定时任务
- 10.2 子 Agent (SubAgent) 系统
- 10.3 定时任务 (Cron) 系统
- 10.4 Heartbeat 心跳服务
- 10.5 三者的协作关系
- 10.6 实战练习
- 10.7 面试高频题
- 10.8 本章小结

10.1 为什么需要子 Agent 和定时任务

10.1.1 主 Agent 的局限

主 Agent 是同步的——用户发消息，Agent 处理，返回结果。但有些场景需要：

场景 1：用户说"帮我调研一下 Rust 和 Go 的对比，要详细的"

- 这可能需要 10 分钟搜索和整理
- 用户不想等 10 分钟什么都做不了

场景 2：用户说"每天早上 8 点给我发一份新闻摘要"

- 需要定时执行，不是一次性对话

场景 3：用户同时提出多个独立任务

- "帮我查天气，同时分析一下这个代码文件"
- 两个任务可以并行处理

10.1.2 解决方案

场景	解决方案
耗时后台任务	SubAgent （子代理后台执行）
定时触发任务	Cron （定时调度）
周期性唤醒	Heartbeat （心跳服务）

10.2 子 Agent (SubAgent) 系统

10.2.1 SpawnTool 启动子代理

SubAgent 通过 spawn 工具启动：

```
{
  "name": "spawn",
  "parameters": {
    "task": "string (必需) - 子代理要执行的任务描述"
  }
}
```

当主 Agent 调用 spawn 时:

主Agent 对话:

User: 帮我详细调研 Rust 和 Go 的对比

Agent: 好的, 这个任务比较耗时, 我启动一个后台任务来做。

[调用工具: spawn]

[task: "详细调研 Rust 和 Go 的对比, 包括性能、生态、学习曲线等维度"]

Agent: 我已经启动了后台调研任务, 完成后会通知你。你可以继续问我其他问题。

(后台 SubAgent 开始独立执行...)

(主Agent 可以继续和用户对话...)

(几分钟后...)

Agent: 后台调研已完成! 以下是 Rust 和 Go 的详细对比...

10.2.2 SubagentManager 管理

SubagentManager 是管理所有子代理生命周期的中心组件:

```
class SubagentManager:
    """ 管理所有后台子代理 """

    def __init__(self, agent_config, message_bus):
        self.config = agent_config
        self.bus = message_bus
        self._active_agents: dict[str, SubAgent] = {}

    async def spawn(self, task: str, parent_session_key: str) -> str:
        """ 启动一个新的子代理 """
        agent_id = generate_unique_id()

        sub_agent = SubAgent(
            agent_id=agent_id,
            task=task,
            config=self._build_subagent_config(),
            parent_session_key=parent_session_key,
            message_bus=self.bus
        )

        self._active_agents[agent_id] = sub_agent

        # 在后台启动执行
        asyncio.create_task(self._run_agent(sub_agent))

        return agent_id

    async def _run_agent(self, agent: SubAgent):
        """ 在后台运行子代理 """
        try:
            result = await agent.execute()
            # 执行完成后, 通过 MessageBus 回报结果
            await self.bus.publish_inbound(InboundMessage(
                channel="internal",
                sender_id="subagent",
                chat_id=agent.parent_session_key,
                content=f"[后台任务完成] {result}",
                media=[],
                metadata={"subagent_id": agent.agent_id},
            ))
```

```

        session_key=agent.parent_session_key
    ))
except Exception as e:
    logger.error(f"SubAgent {agent.agent_id} failed: {e}")
finally:
    del self._active_agents[agent.agent_id]

```

10.2.3 SubAgent 与主 Agent 的区别

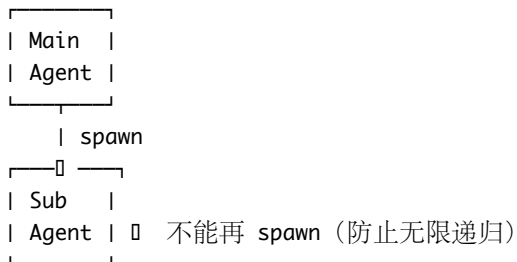
这是面试高频考点：

对比维度	主 Agent	SubAgent（子代理）
迭代限制	40 次	15 次
工具集	完整工具集	受限工具集
MessageTool	可用	不可用
SpawnTool	可用	不可用（防递归）
CronTool	可用	不可用（防递归）
执行方式	前台同步	后台异步
用户交互	直接交互	不能与用户交互
结果汇报	直接返回	通过 MessageBus 回报
生命周期	持续运行	任务完成即终止

10.2.4 为什么限制工具集

防递归原则：

SubAgent 不能调用 spawn → 防止创建子子代理



SubAgent 不能调用 message → 它不直接与用户对话
只能通过 MessageBus 回报结果给主Agent

SubAgent 不能调用 cron → 防止定时任务中再创建定时任务
避免定时任务指数增长

10.2.5 迭代限制的设计思考

主Agent: max_tool_iterations = 40

- └ 理由：主Agent需要处理复杂的多步骤任务
- └ 场景：代码审查 → 搜索 → 修改 → 测试 → 提交
- └ 40 次迭代足以完成大多数复杂任务

SubAgent: max_tool_iterations = 15

- └ 理由：子代理执行的是明确的单一任务
- └ 场景：搜索调研 → 整理 → 汇报
- └ 15 次迭代防止子代理失控消耗过多资源
- └ 如果 15 次不够，说明任务应该拆分

10.2.6 结果回报机制

SubAgent 完成任务后，通过 MessageBus 将结果发送回主 Agent 的会话：

```
SubAgent 执行完成
|
[]
构建 InboundMessage
├─ channel: "internal"          # 内部通道
├─ sender_id: "subagent"       # 标识来自子代理
├─ chat_id: parent_session_key # 发送到父会话
├─ content: "调研结果..."    # 任务结果
└─ metadata: {subagent_id: "xxx"}
|
[]
publish_inbound(message)
|
[]
主Agent 从 Inbound Queue 消费
├─ 识别这是子代理的回报
├─ 将结果整合到当前对话
└─ 向用户展示结果
```

10.2.7 使用场景

场景	适合用 SubAgent	原因
长时间调研	是	不阻塞主对话
批量文件处理	是	后台执行
代码分析	是	耗时但不需要交互
数据采集	是	多源并行采集
实时对话	否	需要即时交互
需要用户确认	否	SubAgent 无法交互

10.3 定时任务（Cron）系统

10.3.1 CronService 基于 APScheduler

Nanobot 的定时任务基于 Python 的 APScheduler（Advanced Python Scheduler）库：

```
from apscheduler.schedulers.asyncio import AsyncIOScheduler

class CronService:
    """ 定时任务管理服务 """

    def __init__(self, message_bus: MessageBus):
        self.scheduler = AsyncIOScheduler()
        self.bus = message_bus
        self._jobs: dict[str, CronJob] = {}

    async def start(self):
        """ 启动调度器 """
        self.scheduler.start()

    async def stop(self):
        """ 停止调度器 """
        self.scheduler.shutdown()
```

10.3.2 cron 工具的三种操作

```
{
  "name": "cron",
  "parameters": {
    "action": "string (必需) - add | list | remove",
    "name": "string - 任务名称 (add/remove 时必需)",
    "schedule": "object - 调度配置 (add 时必需)",
    "message": "string - 触发时发送的消息 (add 时必需)"
  }
}
```

add ——添加定时任务

Agent: [调用 cron]

```
参数: {
  "action": "add",
  "name": "daily_news",
  "schedule": {
    "cron_expr": "0 8 * * *",
    "tz": "Asia/Shanghai"
  },
  "message": "请搜索今天的科技新闻，整理成摘要发给我"
}
```

list ——列出所有任务

Agent: [调用 cron]

```
参数: {
  "action": "list"
}
```

返回:

```
[
  {
    "name": "daily_news",
    "schedule": "0 8 * * * (Asia/Shanghai)",
    "next_run": "2024-03-17 08:00:00",
    "message": "请搜索今天的科技新闻..."
  }
]
```

remove ——删除任务

Agent: [调用 cron]

```
参数: {
  "action": "remove",
  "name": "daily_news"
}
```

10.3.3 调度配置方式

cron 工具支持三种调度配置:

方式一: **every_seconds** (间隔秒数)

```
{
  "schedule": {
    "every_seconds": 3600
  }
}
```

表示每 3600 秒（1 小时）执行一次。

方式二：cron_expr + tz (Cron 表达式)

```
{
  "schedule": {
    "cron_expr": "0 8 * * *",
    "tz": "Asia/Shanghai"
  }
}
```

Cron 表达式格式：

```
_____ 分钟 (0-59)
| _____ 小时 (0-23)
| | _____ 日 (1-31)
| | | _____ 月 (1-12)
| | | | _____ 星期 (0-6, 0=周日)
| | | | |
* * * * *
```

示例：

```
0 8 * * *      → 每天早上 8:00
0 */2 * * *    → 每 2 小时
0 9 * * 1-5    → 工作日 9:00
30 18 * * 5    → 每周五 18:30
0 0 1 * *     → 每月 1 日 0:00
```

方式三：at (指定时间执行一次)

```
{
  "schedule": {
    "at": "2024-03-17T15:30:00",
    "tz": "Asia/Shanghai"
  }
}
```

表示在指定时间执行一次（一次性任务）。

10.3.4 定时任务的执行流程

定时任务触发

```
|
[]
CronService 构建 InboundMessage
├─ channel: "cron"          # 标识来自定时任务
├─ sender_id: "cron_service"
├─ chat_id: original_session_key
├─ content: "请搜索今天的科技新闻..." # 预设的 message
└─ metadata: {cron_job: "daily_news"}
|
[]
publish_inbound(message)
|
[]
Agent 从 Inbound Queue 消费
├─ 像处理普通用户消息一样处理
├─ 执行搜索、整理等操作
└─ 结果通过 OutboundMessage 发送到原始会话
```

10.3.5 防递归机制

Nanobot 严格禁止在定时任务执行上下文中再创建定时任务：

```
class CronTool:
    async def execute(self, action, **kwargs):
        # 检查当前上下文是否来自 Cron 触发
        if self._is_cron_context():
            return "Error: Cannot create cron jobs from within a cron execution context."

        # 正常执行
        ...

    def _is_cron_context(self) -> bool:
        """ 检查当前消息是否来自定时任务触发 """
        current_message = self._get_current_inbound()
        return current_message.channel == "cron"
```

为什么要防递归：

危险场景：

cron_job_A: "每分钟执行，内容是创建一个每分钟执行的cron"

```
t=0: cron_A 触发 → 创建 cron_B (每分钟执行)
t=1: cron_A 触发 → 创建 cron_C
      cron_B 触发 → 创建 cron_D
t=2: cron_A 触发 → 创建 cron_E
      cron_B 触发 → 创建 cron_F
      cron_C 触发 → 创建 cron_G
      cron_D 触发 → 创建 cron_H
```

指数增长 → 系统崩溃！

面试要点：防递归是系统设计中的重要原则。在 SubAgent 中禁止 spawn（防止子代理递归），在 Cron 中禁止创建新 Cron（防止定时任务递归），这些都是对“失控增长”的防御。

10.4 Heartbeat 心跳服务

10.4.1 什么是 Heartbeat

Heartbeat 是一种周期性唤醒 Agent 的机制。与 Cron 不同，Heartbeat 的目的不是执行特定任务，而是让 Agent “保持活跃”并自主决定需要做什么。

Cron: "每天8点，发送今日新闻" → 明确的任务

Heartbeat: "每30分钟醒来，看看有没有事" → 自主判断

10.4.2 配置方式

在 config.json 中配置 Heartbeat：

```
{
  "agents": {
    "defaults": {
      "heartbeat": {
        "enabled": true,
        "interval_s": 1800,
        "keep_recent_messages": 5
      }
    }
  }
}
```

```

    }
}
}

```

字段	类型	说明
enabled	bool	是否启用心跳
interval_s	int	心跳间隔（秒）
keep_recent_messages	int	心跳时保留的最近消息数

10.4.3 心跳触发机制

```
class HeartbeatService:
```

```
    """ 心跳服务 """
```

```

    def __init__(self, config: dict, message_bus: MessageBus):
        self.enabled = config.get("enabled", False)
        self.interval = config.get("interval_s", 1800)
        self.keep_recent = config.get("keep_recent_messages", 5)
        self.bus = message_bus

```

```
    async def start(self):
```

```
        """ 启动心跳循环 """
```

```
        if not self.enabled:
```

```
            return
```

```
        while True:
```

```
            await asyncio.sleep(self.interval)
```

```
            await self._heartbeat()
```

```
    async def _heartbeat(self):
```

```
        """ 执行一次心跳 """
```

```
        # 构建心跳消息
```

```

        message = InboundMessage(
            channel="heartbeat",
            sender_id="heartbeat_service",
            chat_id="heartbeat",
            content="[Heartbeat] 定期检查，请查看是否有待处理事项。",
            media=[],
            metadata={
                "type": "heartbeat",
                "keep_recent_messages": self.keep_recent
            },
            session_key="heartbeat:default"
        )

```

```
        await self.bus.publish_inbound(message)
```

10.4.4 Heartbeat 的使用场景

场景	配置	说明
健康检查	interval_s: 300	每 5 分钟检查系统状态
邮件摘要	interval_s: 3600	每小时检查新邮件
项目监控	interval_s: 1800	每 30 分钟检查项目状态
知识更新	interval_s: 86400	每天更新知识库

10.4.5 keep_recent_messages 的作用

心跳触发时，不需要加载完整的会话历史（太浪费 token），只保留最近的 N 条消息作为上下文：

完整会话历史：100 条消息
keep_recent_messages: 5

心跳时的上下文：
└─ System Prompt（正常）
└─ MEMORY.md（正常）
└─ 最近 5 条消息（而非全部 100 条）

- 好处：
- ▢ 节省 token
 - ▢ Agent 仍能了解最近状态
 - ▢ 减少不必要的信息干扰

10.5 三者的协作关系

10.5.1 SubAgent、Cron、Heartbeat 对比

维度	SubAgent	Cron	Heartbeat
触发方式	主 Agent 调用 spawn	定时表达式	固定间隔
任务内容	明确的单一任务	预设的消息	Agent 自主决定
生命周期	执行完即终止	持续存在	持续运行
与用户交互	不能	结果发到会话	按需交互
并发	可多个并行	独立调度	单实例
防递归	不能再 spawn	不能在 cron 中创建 cron	N/A

10.5.2 协作场景示例

场景：智能助手管理日常工作

Heartbeat（每30分钟唤醒）

- └─ 检查是否有新邮件 → 有 → spawn子Agent处理
- └─ 检查 GitHub PR → 有新PR → 通知用户
- └─ 检查待办事项 → 有到期项 → 提醒用户

Cron（定时任务）

- └─ 每天 8:00 → 发送今日日程
- └─ 每天 18:00 → 发送工作总结
- └─ 每周五 17:00 → 生成周报

SubAgent（按需启动）

- └─ 用户："帮我调研这个技术方案" → spawn后台调研
- └─ Heartbeat发现新邮件 → spawn后台处理

└─ Cron触发周报 → spawn后台收集数据和生成

10.5.3 设计模式分析

SubAgent = Actor 模式

- └─ 独立运行的轻量级进程
- └─ 通过消息传递通信
- └─ 任务完成自动销毁

Cron = Observer 模式（时间驱动）

- └─ 监听时间事件
- └─ 触发时通知 Agent
- └─ 本身不执行业务逻辑

Heartbeat = Polling 模式

- └─ 定期检查
- └─ 让 Agent 自主决策
- └─ 类似于操作系统的定时器中断

10.6 实战练习

练习 1：使用 SubAgent 并行处理

启动 Agent

nanobot

对话示例：

You：帮我做两件事：

1. 搜索 Python 3.12 的新特性

2. 搜索 Rust 2024 的发展趋势

分别用后台任务处理

Agent 应该会调用两次 spawn，启动两个 SubAgent 并行执行

你可以继续与主 Agent 对话，等后台任务完成后收到结果

练习 2：创建定时任务

nanobot

对话示例：

You：帮我创建一个定时任务，每小时提醒我休息一下

Agent 会调用 cron 工具：

{

"action": "add",

"name": "rest_reminder",

"schedule": {"every_seconds": 3600},

"message": " 休息提醒：已经工作 1 小时了，起来活动一下吧！ "

}

查看任务列表

You：列出所有定时任务

删除任务

You：删除休息提醒任务

练习 3: 配置 Heartbeat

```
{
  "agents": {
    "defaults": {
      "heartbeat": {
        "enabled": true,
        "interval_s": 300,
        "keep_recent_messages": 3
      }
    }
  }
}
```

启动后, 每 5 分钟 Agent 会自动醒来
你可以在 AGENTS.md 中告诉它醒来时要做什么:

```
# AGENTS.md 中添加:
# ## 心跳行为
# 当收到心跳唤醒时:
# 1. 检查 workspace 中是否有新文件
# 2. 如果有 TODO.md, 检查待办事项的状态
# 3. 如果发现需要处理的事项, 主动通知用户
```

10.7 面试高频题

题目 1: Nanobot 的 SubAgent 系统是怎么设计的?

参考回答:

“Nanobot 的 SubAgent 通过 spawn 工具启动, 由 SubagentManager 统一管理。

启动机制: 主 Agent 调用 spawn 工具时, SubagentManager 创建一个新的 SubAgent 实例, 并通过 `asyncio.create_task` 在后台运行, 不阻塞主对话。

关键约束: SubAgent 相比主 Agent 有三个限制——迭代次数从 40 降到 15, 防止长时间运行; 不能使用 MessageTool, 因为它不直接与用户交互; 不能使用 SpawnTool 和 CronTool, 防止递归创建。

结果回报: SubAgent 完成后, 将结果封装为 InboundMessage 通过 MessageBus 发回主 Agent 的会话, 主 Agent 再向用户展示。

这种设计的核心思想是受控的并行性——允许后台执行但严格限制能力边界, 避免失控。”

题目 2: 如何防止 Agent 系统中的递归问题?

参考回答:

“Nanobot 在两个维度防止递归:

第一是 **SubAgent** 防递归——子代理的工具集中不包含 spawn 工具, 所以子代理无法创建子子代理。这在 ToolRegistry 层面就做了过滤, 子代理启动时只注册受限的工具子集。

第二是 **Cron** 防递归——在 cron 触发的执行上下文中, cron 工具会检测当前消息的 channel 是否为‘cron’。如果是, 拒绝执行 add 操作。这防止了定时任务中创建新定时任务导致的指数增长。

这两种防递归策略的思路是一致的: 在特定上下文中限制特定工具的可用性。这是最小权限原则在 Agent 系统中的应用。”

题目 3: Cron 和 Heartbeat 有什么区别?

参考回答:

“两者都是定时触发机制，但设计目的不同。

Cron 是任务驱动——它携带明确的 `message`（搜索今日新闻），Agent 按照这个消息执行特定任务。类似于 Linux 的 `crontab`，用户预设好要做什么和什么时候做。

Heartbeat 是状态驱动——它不携带具体任务，而是定期唤醒 Agent 让它自主检查和决策。Agent 醒来后根据当前状态（有没有新邮件？待办事项是否到期？）自主决定做什么或不做什么。

举个类比：Cron 像闹钟（8 点叫我起床），Heartbeat 像保安巡逻（每 30 分钟巡逻一次，看看有没有异常）。

配置上，Heartbeat 还有个 `keep_recent_messages` 参数，只保留最近 N 条消息作为上下文，避免加载完整历史浪费 token。”

题目 4：如果让你设计一个并行任务系统，你会注意什么？

参考回答：

“我会关注四个方面：

1. 资源控制：限制并发子任务的最大数量，设置单个任务的超时和迭代限制。Nanobot 的 15 次迭代限制就是一个好的实践。
2. 递归防护：禁止子任务创建新的子任务，防止任务数量失控。这需要在工具/权限层面做限制，而不是依赖 LLM 的‘自觉’。
3. 结果汇报：子任务完成后要有可靠的结果回传机制。Nanobot 通过 MessageBus 实现，确保结果不丢失。我会考虑加入超时处理——如果子任务超时未完成，主动通知用户。
4. 状态可观测：提供查看所有正在运行的子任务的方式（名称、状态、运行时间），方便用户和运维人员了解系统状态。”

10.8 本章小结

核心概念图

